HAPINN: A Solver-Guided Anchor-Based Framework for Parameterized Differential Equations

Sergo A. Episkoposian[a], Alfred V. Petrosyan[a], Armen S. Yepiskoposyan[b,*]

[a]*Department of Applied Mathematics and Physics, National Polytechnic University of Armenia, Teryan 105, Yerevan 0003, Armenia*
[b]*Institute of Physics, Yerevan State University, Al. Manukyan 1, Yerevan 0003, Armenia*

Abstract

This paper introduces **HAPINN** (*Hybrid Anchor-based Physics-Informed Neural Network*), a solver-guided framework for parameterized differential equations in which a classical numerical solver provides a limited but informative anchor structure and a neural surrogate is trained simultaneously against the governing physics and anchor consistency. The central methodological point is that anchor information is treated not as dense supervision, but as a structural admissibility mechanism that narrows the set of residual-consistent near-minimizers, reduces solution-branch ambiguity, and improves training stability.

The paper makes four main contributions. First, it formulates HAPINN in a unified way for parameterized boundary-value and initial-boundary-value problems. Second, in a coercive elliptic model setting, it proves a projection-based transfer estimate from solver to surrogate and gives finite-dimensional sufficient conditions under which the anchor-residual stability assumption is valid. Third, it provides an explicit analytical model that reveals in closed form how the anchor term acts as a stabilizer and a selector, to-

*Corresponding author

Email addresses: `sergo.episkoposyan@polytechnic.am` (Sergo A. Episkoposian), `petrosyanalfred58@gmail.com` (Alfred V. Petrosyan), `earmen04@gmail.com` (Armen S. Yepiskoposyan)

gether with a transparent decomposition of the total error into numerical, projection, approximation, optimization, and parametric generalization components. Fourth, it interprets a previously published hybrid PINN–TCAD realization for the p–n junction as an application-level instance of the same framework.

No new large-scale empirical campaign is claimed in this article. The purpose of the paper is conceptual, methodological, and theoretical: to isolate, formalize, and justify a solver-guided principle that has already appeared in application-specific form but has not yet been stated as a standalone framework. In this sense, the paper connects general theory with published application-level evidence while clearly separating framework novelty from prior device-specific results.

Keywords: physics-informed neural networks, parameterized PDEs, solver-guided learning, anchor stabilization, projection theory, scientific machine learning, error decomposition, semiconductor modeling

---

1. Introduction

Parameterized differential equations are central to modern scientific computing. They arise in semiconductor modeling, heat transfer, transport problems, electrochemistry, reaction–diffusion systems, inverse problems, uncertainty quantification, and control. In such settings the governing physics is known, yet repeated numerical solution across a parameter range remains expensive.

Classical numerical solvers remain indispensable because of their interpretability, stability, and mature error theory. Neural surrogates, by contrast, provide fast inference after training, but purely data-driven approximations may fail to preserve the structure of the underlying operator. Physics-informed neural networks (PINNs) seek to reduce this gap by enforcing the differential operator inside the loss functional [1, 2]. Even so, pure PINN methods are known to suffer from optimization stiffness, sensitivity to loss

balancing, difficulty on multiscale problems, and ambiguity among residual-consistent approximations.

The present paper studies one particular way of reducing that ambiguity: a classical solver is used not as a competing alternative to the surrogate and not as a generator of a full dense training dataset, but as a source of a *limited anchor structure*. These anchors may consist of pointwise values, local averages, interface traces, or other observation functionals evaluated at selected parameter instances. The surrogate is then trained by combining the PDE residual with the relevant boundary or initial conditions and a distinct anchor consistency term.

We denote this class of methods by

HAPINN = Hybrid Anchor-based Physics-Informed Neural Network.

The purpose of the present paper is not to claim a new application result for one specific physical system. Rather, its purpose is to identify and formalize a general methodological principle that appears in solver-guided hybrid schemes: sparse, reliable solver-side information can restrict the residual-admissible set and thereby improve structural consistency of the learned surrogate.

The central claim is therefore conceptual and mathematical:

> Anchor information in HAPINN is not merely an auxiliary data source; it acts as a structural admissibility mechanism.

This interpretation distinguishes HAPINN both from pure PINN and from ordinary supervised regression on solver-generated outputs.

A simple running example helps fix ideas. Consider the one-dimensional parameterized Poisson problem

$$-\frac{d^2u}{dx^2}(x;\mu) = \mu\pi^2 \sin(\pi x), \qquad x \in (0,1), \qquad u(0;\mu) = u(1;\mu) = 0, \quad (1.1)$$

whose exact solution is $u^\star(x;\mu) = \mu \sin(\pi x)$. A classical solver can provide reliable values or functionals of $u^\star$ at selected parameters, while the surrogate is trained against both the residual and these anchors. This simple model already exhibits the essential compromise between physics fidelity and solver guidance. We return to it in Section 5.

Main contributions

The contributions of the paper are as follows:

1) a unified abstract formulation of HAPINN is given for parameterized boundary-value and initial-boundary-value problems;

2) a canonical hybrid functional is stated, together with a precise methodological distinction from pure PINN, plain regression, and standard multi-fidelity fitting;

3) a coercive elliptic model theory is developed, including an anchor-residual stability assumption, finite-dimensional sufficient conditions, a projection-based transfer theorem, and a consistency corollary;

4) an explicit analytical toy model is derived, showing how the anchor term mediates the compromise between residual fidelity and solver fidelity;

5) a five-component error decomposition is formulated;

6) a previously published hybrid PINN–TCAD realization for the p–n junction is interpreted as a concrete instance of HAPINN.

Scope of the paper

This is a framework paper. It is not a new semiconductor application paper, and it does not claim a new large benchmark campaign. The application-level role of the semiconductor example is narrower and more honest: it serves as published evidence that the solver-guided principle formalized here has already been realized successfully in one concrete PDE setting.

Structure of the paper

Section 2 positions HAPINN in the literature. Section 3 gives the general abstract formulation. Section 4 develops the rigorous model theory. Section 5 presents the analytical toy model. Section 6 gives the error decomposition. Section 7 summarizes the algorithmic structure. Section 8 discusses practical anchor design and loss balancing. Section 9 interprets the published semiconductor realization as a HAPINN instance. Sections 10–12 discuss transferability, limitations, and conclusions.

2. Positioning in the literature

2.1. Pure PINNs

Classical PINNs enforce the differential operator, boundary conditions, and initial data through a composite loss functional [1, 2]. Their strength lies in embedding known physics directly into training. Their weakness is equally well known: optimization pathologies, difficult term balancing, and residual-small approximations that are not always the physically relevant ones.

2.2. Neural operators

Neural operator methods such as DeepONet and the Fourier Neural Operator learn mappings between function spaces rather than solving each instance from scratch [3, 4]. They are powerful for operator learning, but in their standard form they do not impose anchor-based admissibility restrictions inside the loss functional itself.

2.3. Hybrid and stabilized PINN variants

A broad literature now exists on decomposition-based PINNs, gradient-enhanced PINNs, causality-respecting training, and adaptive collocation [5, 6, 7]. These directions address real weaknesses of PINNs, but they do not in general isolate sparse solver-derived observables as a distinct structural mechanism for restricting the residual-consistent set.

2.4. Multi-fidelity learning

Multi-fidelity PINNs combine low- and high-fidelity information in a unified learning architecture [8, 9]. This direction overlaps with HAPINN and must be discussed carefully. The overlap is genuine: both frameworks use solver-side information.

The distinction lies in how that information is interpreted. In multi-fidelity schemes, low-fidelity data are commonly treated as an auxiliary approximation layer, a cheap teacher, or a secondary data source. In HAPINN, by contrast, anchors may be sparse and may be insufficient for stand-alone regression; their role is to eliminate or penalize residual-admissible directions that are incompatible with solver-guided observations. Put differently, multi-fidelity formulations usually use solver information to enrich approximation, whereas HAPINN uses solver information to constrain admissibility.

Proposition 2.1 (Non-equivalence to pure fidelity fitting)**.** *Let X_N be a finite-dimensional approximation class. Suppose that the anchor observations $\{\ell_j(w)\}_{j=1}^m$ do not uniquely determine $w \in X_N$, that is,*

$$\bigcap_{j=1}^{m} \ker(\ell_j|_{X_N}) \neq \{0\},$$

but the combined map

$$w \mapsto \big(A_\mu w, \ell_1(w), \ldots, \ell_m(w)\big)$$

is injective on X_N. Then the corresponding HAPINN objective cannot be reduced to supervised fitting on anchor values alone.

Proof. If the anchor observations do not uniquely determine $w \in X_N$, then anchor loss alone leaves nontrivial undetermined directions in X_N. If the combined residual-anchor map is injective, those directions are removed only when the operator term is included. Hence the hybrid objective is not equivalent to pure fitting on anchor values. □

Proposition 2.1 identifies the core methodological point: in HAPINN, the operator and the anchors act together to define admissibility.

2.5. Historically related reduced-order ideas

Reduced basis and related model reduction methods combine a limited number of high-accuracy solutions with a cheaper approximation space [10, 11]. HAPINN is historically related to that logic, but differs in two essential ways: it operates inside a physics-informed training functional, and it uses anchor-based restriction in a neural approximation setting rather than a classical offline–online decomposition.

2.6. Comparative view

For editorial clarity, Table 1 summarizes the main distinction between neighboring method classes.

Table 1: Positioning of HAPINN relative to neighboring computational paradigms

Method class	**Solver used during training**	**PDE residual in loss**	**Sparse anchors sufficient**	**Admissibility restriction**	**Typical role of solver information**
Pure PINN	No	Yes	Not applicable	No explicit restriction beyond residual terms	None
Regression on solver data	Yes	No	Sometimes	No	Dense or sparse supervision
Multi-fidelity PINN	Yes	Usually yes	Sometimes	Usually weak or indirect	Auxiliary accuracy layer or teacher
Neural operator	Usually no	Usually no	Not central	No	Offline operator learning
Reduced basis / ROM	Yes	Not in training-loss form	Not central	Projection-based admissibility	Basis construction
HAPINN	Yes	Yes	Yes	Yes, explicit	Structural constraint and branch control

3. General HAPINN formulation

3.1. Parameterized PDE problem

Let $\Omega \subset \mathbb{R}^d$ be a bounded domain and let $\mu \in \mathcal{M} \subset \mathbb{R}^p$ denote a parameter. Consider

$$\mathcal{L}_\mu u(x;\mu) = f(x;\mu), \qquad x \in \Omega, \tag{3.1}$$

with boundary condition

$$\mathcal{B}_\mu u(x;\mu) = g(x;\mu), \qquad x \in \partial\Omega, \tag{3.2}$$

and, when relevant, initial condition

$$u(x,0;\mu) = u_0(x;\mu), \qquad x \in \Omega. \tag{3.3}$$

We are interested in the full solution family

$$\{u^\star(\cdot;\mu) \colon \mu \in \mathcal{M}\}.$$

Definition 3.1. The system (3.1)–(3.2), together with (3.3) whenever present, will be called a *parameterized PDE problem.*

3.2. Anchor structure

Let $\mathcal{S}_h$ be a classical solver producing an approximation

$$u_h(\cdot;\mu) = \mathcal{S}_h(\mu).$$

In HAPINN, the solver need not generate a dense training dataset. Instead, it produces an *anchor structure*, which may be pointwise, functional, local, or mixed.

Definition 3.2. An *anchor set* is a collection

$$\mathcal{D}_a = \left\{(\ell_j, \mu^{(k)}, d_{j,k})\right\}, \qquad d_{j,k} = \ell_j\big(u_h(\cdot;\mu^{(k)})\big),$$

where the ℓ_j are chosen observation functionals.

This definition is intentionally broad. It covers point values, traces, averages, integral moments, interface observables, and other physically meaningful quantities. In the abstract formulation of Section 3, the observables may be linear or nonlinear. In the rigorous theory of Section 4, we restrict attention to bounded linear functionals in order to obtain explicit analytic statements.

3.3. Canonical HAPINN loss

Let $u_\theta(x;\mu)$ be the neural surrogate. The canonical hybrid functional is

$$\mathcal{J}(\theta) = \lambda_r \mathcal{J}_r(\theta) + \lambda_b \mathcal{J}_b(\theta) + \lambda_0 \mathcal{J}_0(\theta) + \lambda_a \mathcal{J}_a(\theta) + \lambda_{\mathrm{reg}} \mathcal{J}_{\mathrm{reg}}(\theta), \tag{3.4}$$

where

$$\mathcal{J}_r(\theta) = \frac{1}{|\mathcal{D}_r|} \sum_{(x,\mu)\in\mathcal{D}_r} \left|\mathcal{L}_\mu u_\theta(x;\mu) - f(x;\mu)\right|^2, \tag{3.5}$$

$$\mathcal{J}_b(\theta) = \frac{1}{|\mathcal{D}_b|} \sum_{(x,\mu)\in\mathcal{D}_b} \left|\mathcal{B}_\mu u_\theta(x;\mu) - g(x;\mu)\right|^2, \tag{3.6}$$

$$\mathcal{J}_0(\theta) = \frac{1}{|\mathcal{D}_0|} \sum_{(x,\mu)\in\mathcal{D}_0} \left|u_\theta(x,0;\mu) - u_0(x;\mu)\right|^2, \tag{3.7}$$

$$\mathcal{J}_a(\theta) = \frac{1}{|\mathcal{D}_a|} \sum_{(\ell,\mu,d)\in\mathcal{D}_a} \left|\ell(u_\theta(\cdot;\mu)) - d\right|^2. \tag{3.8}$$

If $\lambda_a = 0$, one recovers an ordinary PINN. If the residual terms are removed and only $\mathcal{J}_a$ remains, one obtains supervised fitting on sparse observations. HAPINN occupies the intermediate regime in which both ingredients are essential.

3.4. Geometric interpretation

Define the residual-admissible set

$$\mathcal{A}_\varepsilon = \{v \in \mathcal{U} : \mathcal{E}_r(v) \le \varepsilon_r,\ \mathcal{E}_b(v) \le \varepsilon_b,\ \mathcal{E}_0(v) \le \varepsilon_0\}.$$

After adding the anchor criterion

$$\mathcal{E}_a(v) = \sum_{j=1}^{m} \left|\ell_j(v) - \ell_j(u_h)\right|^2,$$

one obtains

$$\mathcal{A}_{\varepsilon,\delta} = \{v \in \mathcal{A}_\varepsilon : \mathcal{E}_a(v) \le \delta\} \subseteq \mathcal{A}_\varepsilon.$$

Proposition 3.3 (Anchor-based narrowing)**.** *If the anchor functionals are informative on the effective approximation class, then $\mathcal{A}_{\varepsilon,\delta}$ is strictly smaller than $\mathcal{A}_\varepsilon$ for sufficiently small δ.*

Proof. The inclusion is immediate. If two residual-admissible elements differ in at least one informative observable, they cannot both satisfy the anchor condition for sufficiently small δ. □

Proposition 3.4 (Level-set selection)**.** *Let*

$$\mathcal{J}_{\text{PINN}} = \lambda_r \mathcal{J}_r + \lambda_b \mathcal{J}_b + \lambda_0 \mathcal{J}_0$$

and

$$\mathcal{J}_{\text{HAPINN}} = \mathcal{J}_{\text{PINN}} + \lambda_a \mathcal{J}_a, \qquad \lambda_a > 0.$$

Then for every $\varepsilon > 0$,

$$\Theta^\varepsilon_{\text{HAPINN}} = \{\theta : \mathcal{J}_{\text{HAPINN}}(\theta) \le \varepsilon\} \subseteq \Theta^\varepsilon_{\text{PINN}} = \{\theta : \mathcal{J}_{\text{PINN}}(\theta) \le \varepsilon\},$$

and every element of $\Theta^\varepsilon_{\text{HAPINN}}$ satisfies

$$\mathcal{J}_a(\theta) \le \varepsilon/\lambda_a.$$

Proof. By nonnegativity of $\mathcal{J}_a$,

$$\mathcal{J}_{\mathrm{PINN}}(\theta) \le \mathcal{J}_{\mathrm{HAPINN}}(\theta).$$

If $\theta \in \Theta^{\varepsilon}_{\mathrm{HAPINN}}$, then

$$\lambda_a \mathcal{J}_a(\theta) \le \mathcal{J}_{\mathrm{HAPINN}}(\theta) \le \varepsilon,$$

hence the stated bound. □

4. Rigorous model theory

4.1. Coercive elliptic setting

For the theory we consider a standard coercive elliptic model. Let $\Omega \subset \mathbb{R}^d$ be bounded and Lipschitz, and let

$$V = H^1_0(\Omega).$$

For each $\mu \in \mathcal{M}$ define

$$a_\mu(u, v) = \int_\Omega a(x, \mu) \nabla u \cdot \nabla v \, dx + \int_\Omega c(x, \mu) u \, v \, dx, \tag{4.1}$$

with

$$a(x, \mu) \ge a_0 > 0, \tag{4.2}$$

$$c(x, \mu) \ge 0, \tag{4.3}$$

uniformly in μ, and let $f_\mu \in V'$. The weak solution $u^\star(\cdot; \mu) \in V$ is defined by

$$a_\mu(u^\star, v) = \langle f_\mu, v \rangle, \qquad \forall v \in V. \tag{4.4}$$

Let $A_\mu : V \to V'$ denote the induced operator,

$$\langle A_\mu u, v\rangle = a_\mu(u, v).$$

4.2. Scope of the theory

The analysis in this section is carried out for a linear approximation space $X_N \subset V$ and bounded linear anchor functionals. This is the natural setting for an explicit projection-based theory. For neural network parameterizations, the same ideas should be interpreted locally, through the linearization of the nonlinear approximation manifold around a reference state or a trained surrogate. The genuinely nonlinear neural case is important, but it lies beyond the scope of the present theorem package.

4.3. Surrogate class and model functional

Let $X_N \subset V$ be a finite-dimensional approximation class, and let

$$\ell_1, \dots, \ell_m \in V'$$

be bounded linear anchor observables. For the solver approximation $u_h(\cdot;\mu)$ define

$$d_j^h(\mu) = \ell_j(u_h(\cdot;\mu)).$$

We consider the model hybrid functional

$$\mathcal{J}_\mu^{(N)}(w) = \lambda_r \|A_\mu w - f_\mu\|_{V'}^2 + \lambda_a \sum_{j=1}^{m} \left|\ell_j(w) - d_j^h(\mu)\right|^2, \qquad w \in X_N. \quad (4.5)$$

4.4. Anchor-residual stability

Assumption 4.1 (Anchor-residual stability on X_N)**.** There exists $C_{\mathrm{st}} > 0$ such that for every $v \in X_N$,

$$\|v\|_V \le C_{\mathrm{st}} \left(\|A_\mu v\|_{V'} + \Big(\sum_{j=1}^{m} |\ell_j(v)|^2 \Big)^{1/2} \right). \quad (4.6)$$

This assumption expresses observability on the surrogate class: residual control and anchor observations together define the norm.

Proposition 4.2 (Finite-dimensional sufficient condition)**.** *Suppose that the linear map*

$$T_\mu : X_N \to V' \times \mathbb{R}^m, \qquad T_\mu(v) = \big(A_\mu v, \ell_1(v), \dots, \ell_m(v)\big)$$

is injective. Then Assumption 4.1 holds on X_N.

Proof. Since X_N is finite-dimensional, all norms on X_N are equivalent. Equip $V' \times \mathbb{R}^m$ with the norm

$$\|(F,\alpha)\|_* = \|F\|_{V'} + \Big(\sum_{j=1}^{m} |\alpha_j|^2\Big)^{1/2}.$$

Injectivity of T_μ implies that $\|T_\mu(v)\|_*$ is a norm on X_N. By finite-dimensional norm equivalence, there exists $C_{\mathrm{st}} > 0$ such that

$$\|v\|_V \le C_{\mathrm{st}} \, \|T_\mu(v)\|_* \,,$$

which is exactly (4.6). □

Remark 4.3. Proposition 4.2 turns the abstract stability assumption into a concrete design principle: anchors must eliminate directions that are weakly constrained by the residual on the chosen approximation class.

4.5. Projection-based transfer theorem

Let $\mathrm{P}_N : V \to X_N$ be a stable projector. Since $u_h(\cdot;\mu)$ need not lie in X_N, comparison with the surrogate must pass through the projection.

Theorem 4.4 (Projection-based transfer from solver to surrogate)**.** *Assume*

Assumption 4.1. Then for every $w_N \in X_N$,

$$\|w_N - \mathrm{P}_N\, u_h(\cdot;\mu)\|_V \le C_{\mathrm{st}} \left(\|A_\mu(w_N - \mathrm{P}_N\, u_h(\cdot;\mu))\|_{V'} + \Delta_a(w_N, \mathrm{P}_N\, u_h) \right), \tag{4.7}$$

where

$$\Delta_a(w_N, \mathrm{P}_N\, u_h) = \Big(\sum_{j=1}^{m} |\ell_j(w_N) - \ell_j(\mathrm{P}_N\, u_h(\cdot;\mu))|^2 \Big)^{1/2}.$$

Consequently,

$$\begin{aligned} \|w_N - u_h(\cdot;\mu)\|_V \le{}& \|u_h(\cdot;\mu) - \mathrm{P}_N\, u_h(\cdot;\mu)\|_V \\ &+ C_{\mathrm{st}} \Big(\|A_\mu w_N - f_\mu\|_{V'} + \|A_\mu\, \mathrm{P}_N\, u_h(\cdot;\mu) - f_\mu\|_{V'} + \Delta_a(w_N, \mathrm{P}_N\, u_h) \Big). \end{aligned} \tag{4.8}$$

Proof. Set

$$v_N = w_N - \mathrm{P}_N\, u_h(\cdot;\mu) \in X_N.$$

Applying Assumption 4.1 to v_N gives

$$\|v_N\|_V \le C_{\mathrm{st}} \left(\|A_\mu v_N\|_{V'} + \Big(\sum_{j=1}^{m} |\ell_j(v_N)|^2 \Big)^{1/2} \right).$$

Since

$$A_\mu v_N = A_\mu w_N - A_\mu\, \mathrm{P}_N\, u_h(\cdot;\mu),$$

we obtain

$$\|A_\mu v_N\|_{V'} \le \|A_\mu w_N - f_\mu\|_{V'} + \|A_\mu\, \mathrm{P}_N\, u_h(\cdot;\mu) - f_\mu\|_{V'}.$$

Likewise,

$$\ell_j(v_N) = \ell_j(w_N) - \ell_j(\mathrm{P}_N\, u_h(\cdot;\mu)).$$

Substituting these relations yields (4.7). The triangle inequality then gives (4.8). □

Remark 4.5. The theorem shows that surrogate error should be measured against $\mathrm{P}_N\, u_h$, not directly against u_h. The projection-approximation term is intrinsic unless $u_h \in X_N$.

4.6. Consistency

Corollary 4.6 (Consistency of the hybrid scheme in the model setting)**.** *Suppose that:*

1) $\|u^\star(\cdot;\mu) - u_h(\cdot;\mu)\|_V \to 0$ *as* $h \to 0$*;*

2) $\|u_h(\cdot;\mu) - \mathrm{P}_N\, u_h(\cdot;\mu)\|_V \to 0$ *as* $N \to \infty$*;*

3) there exists a sequence $w_{N,h} \in X_N$ *such that*

$$\|A_\mu w_{N,h} - f_\mu\|_{V'} \to 0, \qquad \Delta_a(w_{N,h}, \mathrm{P}_N\, u_h) \to 0;$$

4) Assumption 4.1 holds uniformly over the relevant regime.

Then

$$\|w_{N,h} - u^\star(\cdot;\mu)\|_V \to 0$$

under a coordinated limit $h \to 0$, $N \to \infty$.

Proof. By Theorem 4.4,

$$\|w_{N,h} - u_h(\cdot;\mu)\|_V \to 0.$$

Hence

$$\|w_{N,h} - u^\star(\cdot;\mu)\|_V \le \|w_{N,h} - u_h(\cdot;\mu)\|_V + \|u_h(\cdot;\mu) - u^\star(\cdot;\mu)\|_V \to 0.$$

□

Remark 4.7. Condition 3 is an approximation hypothesis, not a hidden theorem. For nonlinear neural parameterizations it is nontrivial, and the present framework does not claim more.

5. Analytical toy model

To see explicitly how anchors affect the optimizer, consider

$$-u''(x;\mu) = \mu\pi^2 \sin(\pi x), \qquad x \in (0,1), \qquad u(0;\mu) = u(1;\mu) = 0, \tag{5.1}$$

whose exact solution is $u^\star(x;\mu) = \mu \sin(\pi x)$.

Let the surrogate class be

$$u_\alpha(x;\mu) = \alpha(\mu)\sin(\pi x),$$

and let the anchor solver produce

$$u_h(x;\mu) = a_h(\mu)\sin(\pi x).$$

Consider

$$\mathcal{J}_{\text{toy}}(\alpha) = \frac{\lambda_r \pi^4}{2} \left|\alpha(\mu) - \mu\right|^2 + \lambda_a \left|\alpha(\mu) - a_h(\mu)\right|^2. \tag{5.2}$$

The factor $\pi^4/2$ comes from the squared $L^2(0,1)$ residual of the one-mode ansatz: since

$$-u_\alpha'' - \mu\pi^2 \sin(\pi x) = \pi^2(\alpha - \mu)\sin(\pi x),$$

one has

$$\int_0^1 \left|-u_\alpha'' - \mu\pi^2 \sin(\pi x)\right|^2 dx = \pi^4(\alpha-\mu)^2 \int_0^1 \sin^2(\pi x)\, dx = \frac{\pi^4}{2}(\alpha-\mu)^2.$$

Theorem 5.1 (Closed-form anchor compromise)**.** *The functional* (5.2) *has the unique minimizer*

$$\alpha_{\text{opt}}(\mu) = \frac{\lambda_r \pi^4 \mu + 2\lambda_a a_h(\mu)}{\lambda_r \pi^4 + 2\lambda_a}. \tag{5.3}$$

Hence

$$\left|\alpha_{\text{opt}}(\mu) - \mu\right| = \frac{2\lambda_a}{\lambda_r \pi^4 + 2\lambda_a} \left|a_h(\mu) - \mu\right|, \tag{5.4}$$

and

$$|\alpha_{\mathrm{opt}}(\mu) - a_h(\mu)| = \frac{\lambda_r \pi^4}{\lambda_r \pi^4 + 2\lambda_a} |a_h(\mu) - \mu| . \tag{5.5}$$

Proof. Differentiate (5.2) with respect to α:

$$\frac{d}{d\alpha}\mathcal{J}_{\mathrm{toy}} = \lambda_r \pi^4(\alpha - \mu) + 2\lambda_a(\alpha - a_h).$$

Setting the derivative equal to zero gives (5.3). The remaining identities follow by subtraction. □

Corollary 5.2 (Interpretation of the one-mode mechanism)**.** *The minimizer exhibits the following limiting regimes:*

i) ***Physics limit:*** *if $\lambda_a \to 0$, then $\alpha_{\mathrm{opt}}(\mu) \to \mu$;*

ii) ***Anchor limit:*** *if $\lambda_r \to 0$, then $\alpha_{\mathrm{opt}}(\mu) \to a_h(\mu)$;*

iii) ***Damped solver error:*** *the deviation from the exact coefficient is a weighted fraction of the solver error;*

iv) ***Selector effect:*** *the anchor term suppresses admissible directions that are not aligned with the solver-guided coefficient.*

Remark 5.3. The theorem does not prove every claim one might want about nonlinear branch selection, but it makes one point exact: the anchor term does not replace physics; it changes the geometry of minimization.

6. Error decomposition

Let $u^\star$ be the exact solution, u_h the anchor solver solution, and u_θ the trained HAPINN surrogate. Then

$$u^\star - u_\theta = (u^\star - u_h) + (u_h - u_\theta), \tag{6.1}$$

hence

$$\|u^\star - u_\theta\|_{\mathcal{U}} \le \|u^\star - u_h\|_{\mathcal{U}} + \|u_h - u_\theta\|_{\mathcal{U}} . \tag{6.2}$$

Introducing the projection onto the surrogate class,

$$u_h - u_\theta = (u_h - \mathrm{P}_N\, u_h) + (\mathrm{P}_N\, u_h - u_\theta),$$

one obtains

$$\|u^\star - u_\theta\|_{\mathcal{U}} \le \underbrace{\|u^\star - u_h\|_{\mathcal{U}}}_{\text{numerical error}} + \underbrace{\|u_h - \mathrm{P}_N\, u_h\|_{\mathcal{U}}}_{\text{projection error}} + \underbrace{\|\mathrm{P}_N\, u_h - u_\theta\|_{\mathcal{U}}}_{\text{training/transfer error}} . \tag{6.3}$$

Refining further with the best attainable surrogate in the class gives

$$\|u^\star - u_\theta\|_{\mathcal{U}} \le \underbrace{\|u^\star - u_h\|_{\mathcal{U}}}_{\text{numerical}} + \underbrace{\|u_h - \mathrm{P}_N\, u_h\|_{\mathcal{U}}}_{\text{projection}} + \underbrace{\inf_{\vartheta} \|\mathrm{P}_N\, u_h - u_\vartheta\|_{\mathcal{U}}}_{\text{approximation}} + \underbrace{\|u_{\vartheta^\star} - u_\theta\|_{\mathcal{U}}}_{\text{optimization}} + \underbrace{\mathcal{E}_{\text{gen}}}_{\text{parameter generalization}} \tag{6.4}$$

where $\mathcal{E}_{\text{gen}}$ denotes the parameter generalization component.

Remark 6.1. This five-way decomposition is methodologically useful because it separates the effect of anchors from the classical numerical error and from the training error of the neural surrogate.

7. Algorithmic framework

Algorithm 1 General HAPINN training loop

```
Specify the parameterized PDE problem and the training parameter set
Choose a base solver $\mathcal{S}_h$ and a surrogate $u_\theta(x;\mu)$
Select anchor parameters and construct an initial anchor set $\mathcal{D}_a$
Build residual, boundary, and initial collocation sets $\mathcal{D}_r, \mathcal{D}_b, \mathcal{D}_0$
Initialize $\lambda_r, \lambda_b, \lambda_0, \lambda_a$
for $t = 1, 2, \ldots, T$ do
    Update $\theta$ by minimizing (3.4)
    Evaluate block losses $\mathcal{J}_r, \mathcal{J}_b, \mathcal{J}_0, \mathcal{J}_a$
    if high residual, training drift, or visible instability is detected then
        Enrich $\mathcal{D}_a$ with new anchor observations
    end if
    Optionally rebalance the weights by a standard strategy
end for
Validate on unseen parameters and report both accuracy and stability
```

The algorithm is intentionally stated at the framework level. Its role is to make the computational logic explicit, not to overstate problem-independent hyperparameter prescriptions.

8. Practical anchor design and loss balancing

8.1. Anchorization regimes

In practice it is useful to distinguish:

1) **full anchorization:** dense solver nodes are used;

2) **sparse anchorization:** selected anchor observables only;

3) **local anchorization:** anchors are concentrated in regions with layers, interfaces, or steep gradients;

4) **adaptive anchorization:** new anchors are added during training.

8.2. Residual-informed anchor indicator

A practical residual-informed anchor indicator may be defined as

$$\eta(x,\mu) = \omega_r\,\rho_r(x,\mu) + \omega_s\,\rho_s(x,\mu) + \omega_g\,\rho_g(x,\mu),$$

where ρ_r denotes a local residual magnitude, ρ_s a local stiffness or sensitivity indicator, and ρ_g the gradient magnitude or an interface detector. This indicator is intentionally generic: it is meant to describe a family of residual-informed enrichment strategies rather than a single universal rule.

8.3. Loss balancing

The hybrid weights are crucial. In practice, four regimes are natural:

1) fixed weights when the PDE is simple and scaling is controlled;
2) staged training, with larger early anchor weight followed by stronger residual enforcement;
3) gradient-norm balancing;
4) multi-task weighting borrowed from standard optimization heuristics.

We intentionally avoid prescribing a new universal update law here. In applied settings, well-known balancing strategies from multi-task learning and PINN practice, such as gradient-based normalization or uncertainty-based weighting, may be used when appropriate. The point of the present framework is not to introduce a new balancing heuristic, but to clarify the structural role of the anchor term.

8.4. How many anchors are needed

No universal scaling law exists. In practice one should take into account:

1) smoothness of the map $\mu \mapsto u^\star(\cdot;\mu)$;
2) effective dimension of the solution manifold;

3) degree of nonlinearity and regime transition;

4) quality of the base solver;

5) richness of the surrogate class.

Remark 8.1. The present paper does not claim a new universal hyperparameter rule. The practical guidance here is intentionally moderate and framework-oriented.

9. Published semiconductor realization as a concrete HAPINN instance

The framework introduced here is new, but the general principle is not merely hypothetical. A published hybrid PINN–TCAD study on p–n junction simulation provides a concrete application-level instance of the same architecture [16].

Proposition 9.1. *The published hybrid PINN–TCAD scheme for p–n junction simulation is a special case of HAPINN, with solver outputs acting as anchors and the data-consistency block acting as $\mathcal{J}_a$.*

Proof. The published scheme minimizes a composite functional of the form

$$L = w_{\mathrm{PDE}} L_{\mathrm{PDE}} + w_{\mathrm{BC}} L_{\mathrm{BC}} + w_{\mathrm{data}} L_{\mathrm{data}},$$

where L_{data} is computed from solver-generated quantities. This matches the HAPINN structure by direct identification of terms. □

The role of this observation is interpretive, not duplicative. The present paper does not claim the semiconductor experiment as a new result. What it claims is narrower and cleaner: the published application fits the framework, and the framework helps explain why that class of hybrid solvers can be effective.

Table 2: Empirical evidence from the published semiconductor study

Scenario	**Reported result**	**Framework interpretation**
1D potential prediction across silicon devices	0.48% RMS error	Solver-guided surrogate reaches sub-percent fidelity
Acceleration vs. Sentaurus TCAD	47× speed-up	Expensive repeated solves are replaced by learned inference
2D extension	0.62% RMS error, 12× acceleration	The anchor-guided idea is not restricted to the minimal 1D setting
C–V validation	0.9% RMS deviation, 177× runtime reduction	Solver-guided training remains consistent with an additional device-level observable
Spillover effect quantification	up to 8 mV built-in potential correction	Hybrid framework reveals physically meaningful deviation from the analytical approximation

What is new here relative to the published semiconductor paper

To avoid conceptual confusion, let us state the distinction explicitly.

In the published semiconductor paper, the main object was the p–n junction problem itself, together with numerical validation and device-level implications. In the present paper, the main object is the *general solver-guided anchor-based framework*. The novelty of the present paper lies in:

a) unified terminology and abstract formulation;

b) canonical hybrid loss structure;

c) projection-based transfer theory;

d) consistency result in a model elliptic setting;

e) explicit analytical anchor mechanism;

f) general error decomposition;

g) formal translation of the application-level hybrid scheme into the language of a broader method class.

This article therefore does not repeat the semiconductor paper; it extracts its general methodological invariant.

10. Problem classes and transferability

The framework applies most directly to coercive elliptic problems, which is precisely the setting of the rigorous theory developed above. It also extends naturally to parabolic and evolutionary equations by inclusion of the initial-loss term and time as an additional variable in the surrogate.

For coupled systems, anchors may be specified for all components, only for the stiffest components, or on selected subdomains. This is especially relevant for drift–diffusion–Poisson systems, electro-thermal couplings, and other multiphysics models.

In nonlinear and multi-regime problems, the role of anchors becomes especially important because residual-based training may admit multiple near-admissible branches. In such settings, HAPINN is best understood as a framework for preserving solver-guided branch consistency.

In inverse problems and parameter identification, the anchor module can reduce the set of nonphysical candidate solutions and thereby improve robustness of joint optimization over state and parameter variables.

11. Limitations and scope

HAPINN is not a universal automatic solution to all problems in scientific machine learning.

1) **Solver quality matters.** If the numerical solver is coarse or biased, anchors can transmit that bias.

2) **Coverage matters.** Sparse anchors do not by themselves guarantee broad parametric generalization.

3) **Very high-dimensional and stiff regimes remain difficult.** Anchors help, but they do not remove the curse of dimensionality or all optimization pathologies.

4) **The present theory is intentionally limited.** The rigorous results are proved only in a coercive elliptic model setting.

5) **The linear theory does not by itself settle the nonlinear neural case.** For neural parameterizations, the theorems above should be read as local structural guidance rather than as full global convergence theorems.

6) **Solver bias can propagate.** If the base solver has a systematic bias that does not vanish under refinement, then HAPINN may inherit that bias, modulated but not eliminated by the anchor weight.

7) **This paper is not an empirical benchmark paper.** It does not claim a new comparative numerical campaign across multiple PDE families.

The last point is especially important. The contribution of this article is conceptual, methodological, and theoretical. The application-level evidence it uses is published evidence, not newly claimed unpublished experimentation.

12. Conclusion

This paper introduced HAPINN, a solver-guided anchor-based framework for parameterized differential equations. Its central idea is simple but struc-

turally important: solver information need not be dense, and it need not act merely as an auxiliary fidelity layer. Used sparsely and deliberately, anchor observations can alter the admissible geometry of residual-based training.

The paper established this claim at four levels:

1) **framework level:** a unified hybrid formulation was given for parameterized PDEs;

2) **theory level:** a projection-based transfer estimate and finite-dimensional sufficient conditions for anchor-residual stability were proved in a coercive elliptic setting;

3) **analytical level:** a closed-form toy model showed how anchors mediate the compromise between physics fidelity and solver guidance;

4) **application level:** a published hybrid PINN–TCAD realization was interpreted as a concrete HAPINN instance.

HAPINN should therefore be understood not as a minor variant of PINN, but as a distinct solver-guided physics-informed framework. The next natural steps are broader benchmark studies, sharper analysis of anchor informativeness, nonlinear and coupled extensions, uncertainty-aware multi-anchor variants, and larger reproducibility packages. Even in its present form, however, the framework provides a clearer language and a more explicit mathematical backbone for a class of methods that has so far been described mostly in application-specific terms.

Acknowledgements

The authors thank colleagues and collaborators for discussions on hybrid scientific machine learning, numerical approximation, and semiconductor modeling.

Data availability

No new dataset was generated for this framework article. The application-level evidence discussed in Section 9 comes from a previously published study, where implementation code, datasets, and reproduction scripts are reported as openly available.

Code availability

This article does not introduce a new benchmark code package of its own. Code availability for the semiconductor realization discussed in Section 9 is governed by the published application paper.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Raissi, M., Perdikaris, P., Karniadakis, G. E.: Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, **378**, 686–707 (2019)

[2] Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., Yang, L.: Physics-informed machine learning. *Nature Reviews Physics*, **3**, 422–440 (2021)

[3] Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G. E.: Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, **3**, 218–229 (2021)

[4] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., Anandkumar, A.: Fourier neural operator for parametric partial differential equations. *International Conference on Learning Representations* (2021)

[5] Meng, X., Li, Z., Zhang, D., Karniadakis, G. E.: PPINN: Parareal physics-informed neural network for time-dependent PDEs. *Computer Methods in Applied Mechanics and Engineering*, **370**, 113250 (2020)

[6] Yu, J., Lu, L., Meng, X., Karniadakis, G. E.: Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. *Computer Methods in Applied Mechanics and Engineering*, **393**, 114823 (2022)

[7] Wang, S., Sankaran, S., Perdikaris, P.: Respecting causality is all you need for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, **399**, 115295 (2022)

[8] Penwarden, M., Zhe, S., Narayan, A., Kirby, R. M.: Multifidelity modeling for physics-informed neural networks. *Journal of Computational Physics*, **451**, 110844 (2022)

[9] Xiao, H., Lu, D., Owhadi, H., et al.: Multi-fidelity scientific machine learning for PDEs: recent advances and open problems. *Archives of Computational Methods in Engineering*, **31**, 1–32 (2024)

[10] Quarteroni, A., Rozza, G.: *Reduced Order Methods for Modeling and Computational Reduction*. Springer, Milano (2014)

[11] Benner, P., Gugercin, S., Willcox, K.: A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, **57**(4), 483–531 (2015)

[12] Evans, L. C.: *Partial Differential Equations*. American Mathematical Society, Providence (2010)

[13] Quarteroni, A., Sacco, R., Saleri, F.: *Numerical Mathematics*. Springer, Berlin (2007)

[14] Brenner, S. C., Scott, R.: *The Mathematical Theory of Finite Element Methods*. Springer, New York (2008)

[15] Goodfellow, I., Bengio, Y., Courville, A.: *Deep Learning*. MIT Press, Cambridge (2016)

[16] Petrosyan, A. V., Yepiskoposyan, A. S.: Hybrid PINN–TCAD framework for sub-percent p–n junction simulation with spillover error quantification. *Journal of Computational Electronics*, **25**, Article 12 (2026)